redrob | H2S

INDIA.RUNS

Build what next India runs on

Team Name : InnoQueens

Team Leader Name : Jidnyasa Patil

Problem Statement : AI-Powered Intelligent Candidate Ranking System

Problem & Solution Fit

Recruiters need a trusted shortlist, not a keyword-matched pile.

The hiring gap

Hundreds of profiles can contain the right keywords while still missing the real role fit. A strong recruiter looks beyond tokens: experience depth, production context, evidence, location, responsiveness, and risk signals.

- Keyword-only filters over-reward skill stuffing.
- Manual review is slow and inconsistent at scale.

ContextRank AI response

A hybrid CPU-only ranker that uses local TF-IDF relevance, JD-specific rules, behavioral signals, and profile integrity checks to rank candidates in a way recruiters can defend.

understands role intent

scores full profile

exports top 100

Solution Overview

1

Local text relevance

TF-IDF 1-2 grams over profile text, compared with JD-derived ideal and targeted sub-queries.

2

Structured JD fit

Rules for titles, 5-9 year band, product experience, location fit, and disqualifier patterns.

3

Behavioral modifier

Uses Redrob signals: activity recency, open-to-work, response rate, notice period, verified data.

4

Integrity defense

Hard-penalizes impossible profiles, expert-with-0-duration skills, overlaps, and keyword stuffing.

Final score = (0.40 x TF-IDF + 0.45 x rules + 0.15 baseline) x behavior x honeypot penalty

JD Understanding & Candidate Signals

The system translates role intent into measurable evidence.

Role intent extracted from the JD

- ranking & retrieval systems
- semantic search / embeddings
- RAG, BM25, Elasticsearch
- production ML engineering
- 5-9 years, Pune/Noida preferred

→ match intent to evidence

Candidate evidence evaluated

- headline + summary + career history
- skills with surrounding profile context
- career stability and seniority band
- behavioral response/readiness signals
- integrity checks against honeypots

Ranking Methodology

Retrieval, scoring, and ranking are fused into one reproducible pipeline.

1. **Load JSONL** — candidate profiles
2. **Text relevance** — local TF-IDF cosine
3. **Rule fit** — JD-specific heuristics
4. **Modifiers** — behavior + penalties
5. **Export** — top-100 CSV/XLSX

Tie-breaking and final ordering are deterministic: candidates are sorted by descending score, with stable secondary ordering by candidate_id when scores tie.

Explainability & Data Validation

Every ranked row includes a grounded recruiter-facing reason.

Grounded reasoning

Reasoning is built from actual profile fields: title, company, years, location, core skills, recruiter response rate, and last activity. This avoids generic praise and supports manual review.

Honeypot defense

The ranker checks for mismatched experience totals, expert skills with near-zero duration, overlapping dates, degenerate text, and extreme skill stuffing. Flagged profiles are heavily down-ranked.

Format checks

The generated output keeps exactly 100 rows, ranks 1-100 once each, unique candidate IDs, monotonic scores, and the required columns: candidate_id, rank, score, reasoning.

System Architecture

CPU-only, no-network ranking designed to reproduce inside the validator sandbox.

candidates .jsonl
raw profiles + signals

→

Text builder
headline, summary, history, skills

→

TF-IDF matrix
fit locally at rank time

→

Rule scorer
JD fit + penalties

→

submission.csv
ranked top 100

Why this architecture works ?

- No hosted LLM calls, GPU, or model downloads during ranking.
- TF-IDF provides fast lexical-semantic coverage for retrieval/search roles.
- Rules inject recruiter judgment and prevent obvious bad fits.

Reproduce command

```
python rank.py --candidates
./candidates.jsonl --out
./submission.csv
```

Results & Performance

The submitted output is validator-ready and aligned to the scoring focus.

100 ranked candidates

0.9810 top score

45-90s CPU full-pool target

Top recommended candidates

Rank	Candidate	Score	Why it ranked highly
1	CAND_0055905	0.9810	Senior Machine Learning Engineer at Flipkart, 8.1 yrs, based in London; core skills: Elasticsearch, Embeddings, Fine-tuning LLMs; recruiter respons...
2	CAND_0046525	0.9795	Senior Machine Learning Engineer at Genpact AI, 6.1 yrs, based in Pune, Maharashtra; core skills: Elasticsearch, LangChain, Qdrant; recruiter respo...
3	CAND_0046064	0.9683	Senior NLP Engineer at Salesforce, 8.9 yrs, based in Coimbatore, Tamil Nadu; core skills: BM25, Elasticsearch, OpenSearch; recruiter response rate ...
4	CAND_0011687	0.9295	Senior NLP Engineer at Niramai, 7.8 yrs, based in Indore, Madhya Pradesh; core skills: Embeddings, FAISS, LangChain; recruiter response rate 0.89; ...
5	CAND_0071974	0.8828	Senior AI Engineer at Netflix, 7.8 yrs, based in Vizag, Andhra Pradesh; core skills: BM25, Elasticsearch, Embeddings; recruiter response rate 0.76;...

Technologies Used

Python 3

single reproducible rank.py entry point

scikit-learn

TF-IDF vectorization and cosine similarity

CSV / XLSX

submission-ready ranked output

Local heuristics

JD fit, penalties, behavioral modifiers

Google Colab

small-sample sandbox notebook

GitHub

code repo and reproducibility commands

Submission Assets

Github repo : https://github.com/Jidnyasa-P/AI-Powered-candidate-ranking-system

Ranked Output File (XLSX) :
https://docs.google.com/spreadsheets/d/1Dti2Xj8TvqIqwixHyRouEDZAwVZ8y1B_kuSpie06RvE/edit?usp=sharing

Sandbox / demo link

Colab URL:

https://colab.research.google.com/drive/12BGX0FTISNvTZ36XH5qrg-88evdf7H3y?usp=sharing

The notebook accepts a small candidate sample or uses built-in samples, runs the CPU-only ranker end-to-end, and produces ranked_output.csv within the 5-minute budget.

INDIA.RUNS

Build what next India runs on

THANK YOU